Process Synchronization

Learn about locks in xv6

Contents

- Locks
  - Why Lock?
  - xv6 Lock Implementation
    - atomic operation
    - interrupt while locking
    - memory barrier
  - Key Considerations in Using Locks
- Hands-on Labs
  - FCFS scheduler

Locks

Why Lock?

Race Condition

- Any code that accesses shared data concurrently is likely to yield incorrect results

The section of code where shared resources are accessed is called the **critical section**

```
$ vim kernel/proc.c

 92  int
 93  allocpid()
 94  {
 95    int pid;
...
 98    pid = nextpid;
 99    nextpid = nextpid + 1;
...
102   return pid;
103 }
```

Race Condition Example

Thread 1	Thread 2		Integer value
			0
read value		←	0
increase value			0
write back		→	1
	read value	←	1
	increase value		1
	write back	→	2

Correct result

Thread 1	Thread 2		Integer value
			0
read value		←	0
	read value	←	0
increase value			0
	increase value		0
write back		→	1
	write back	→	1

Incorrect result

What is Lock?

- A **lock** is a synchronization mechanism that enforces mutual exclusion
- **Only one CPU** can execute the critical section at a time
- Locks prevent race conditions on shared data

Lock Example

With lock, **only one CPU can execute** the critical section at a time

```
$ vim kernel/proc.c

 92  int
 93  allocpid()
 94  {
 95    int pid;
...
 97    acquire(&pid_lock);
 98    pid = nextpid;
 99    nextpid = nextpid + 1;
100    release(&pid_lock);
...
102   return pid;
103 }
```

Locks

xv6 Lock Implementation

Naïve Lock Implementation

```
$ vim kernel/my_lock.c

 21  void
 22  acquire(struct spinlock *lk)
 23  {
 24    for (;;) {
 25      if (lk->locked == 0) {
 26        lk->locked = 1;
 27        break;
 28      }
 29    }
 30  }
...
 46  void
 47  release(struct spinlock *lk)
 48  {
 49    lk->locked = 0;
 50  }
```

```
$ vim kernel/spinlock.h

  1  // Mutual exclusion lock.
  2  struct spinlock {
  3    uint locked;       // Is the lock held?
  4
  5    // For debugging:
  6    char *name;        // Name of lock.
  7    struct cpu *cpu;   // The cpu holding the lock.
  8  };
```

Problem: Not Atomic

CPU1

CPU2

```
$ vim kernel/my_lock.c

 21  void
 22  acquire(struct spinlock *lk)
 23  {
 24    for (;;) {
 25      if (lk->locked == 0) {
 26        lk->locked = 1;
 27        break;
 28      }
 29    }
 30  }
...
 46  void
 47  release(struct spinlock *lk)
 48  {
 49    lk->locked = 0;
 50  }
```

```
$ vim kernel/spinlock.h

  1  // Mutual exclusion lock.
  2  struct spinlock {
  3    uint locked;       // Is the lock held?
  4
  5    // For debugging:
  6    char *name;        // Name of lock.
  7    struct cpu *cpu;   // The cpu holding the lock.
  8  };
```

Problem: Not Atomic

CPU1

CPU2

```
$ vim kernel/my_lock.c

 21  void
 22  acquire(struct spinlock *lk)
 23  {
 24    for (;;) {
 25      if (lk->locked == 0) {
 26        lk->locked = 1;
 27        break;
 28      }
 29    }
 30  }
...
 46  void
 47  release(struct spinlock *lk)
 48  {
 49    lk->locked = 0;
 50  }
```

```
$ vim kernel/spinlock.h

  1  // Mutual exclusion lock.
  2  struct spinlock {
  3    uint locked;       // Is the lock held?
  4
  5    // For debugging:
  6    char *name;        // Name of lock.
  7    struct cpu *cpu;   // The cpu holding the lock.
  8  };
```

Problem: Not Atomic

CPU1

CPU2

```
$ vim kernel/my_lock.c

 21  void
 22  acquire(struct spinlock *lk)
 23  {
 24    for (;;) {
 25      if (lk->locked == 0) {
 26        lk->locked = 1;
 27        break;
 28      }
 29    }
 30  }
...
 46  void
 47  release(struct spinlock *lk)
 48  {
 49    lk->locked = 0;
 50  }
```

- Two different CPUs can both see `locked==0` and set `locked=1`
- Lines 25 and 26 must execute atomically

```
$ vim kernel/spinlock.h

  1  // Mutual exclusion lock.
  2  struct spinlock {
  3    uint locked;       // Is the lock held?
  4
  5    // For debugging:
  6    char *name;        // Name of lock.
  7    struct cpu *cpu;   // The cpu holding the lock.
  8  };
```

Atomic Instructions in xv6 Code

- **Atomic instruction**: read the old value and set a new value **in a single operation**
- Hardware guarantees no other CPU can interleave
  - **__sync_lock_test_and_set(&lk->locked, 1)**

```
// pseudo code of __sync_lock_test_and_set

1  int __sync_lock_test_and_set(int *addr, int newval) {
2    int old = *addr;
3    *addr = newval;
4    return old;
5  }
```

  - **__sync_lock_release(&lk->locked)**

```
// pseudo code of __sync_lock_release

1  void __sync_lock_release(int *addr) {
2    *addr = 0;
3  }
```

Atomic Instructions in xv6 Code

- **Atomic instruction**: read the old value and set a new value **in a single operation**
- Hardware guarantees no other CPU can interleave

```
$ vim kernel/spinlock.c

 21  void
 22  acquire(struct spinlock *lk)
 23  {
 24    push_off(); // disable interrupts to avoid deadlock.
 25    if(holding(lk))
 26      panic("acquire");
...
 32    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
 33      ;
...
 39    __sync_synchronize();
...
 42    lk->cpu = mycpu();
 43  }
```

```
$ vim kernel/spinlock.c

 46  void
 47  release(struct spinlock *lk)
 48  {
 49    if(holding(lk))
 50      panic("release");
 51
 52    lk->cpu = 0;
...
 60    __sync_synchronize();
...
 69    __sync_lock_release(&lk->locked);
 70
 71    pop_off();
 72  }
```

Interrupt while Locking

- What if an interrupt occurs while holding a lock?

```
$ vim kernel/sysproc.c

 67  uint64
 68  sys_pause(void)
 69  {
...
 76    acquire(&tickslock);
 77    ticks0 = ticks;
...
 85    release(&tickslock);
 86    return 0;
 87  }
```

```
$ vim kernel/trap.c

164  void
165  clockintr()
166  {
167    if(cpuid() == 0){
168      acquire(&tickslock);
169      ticks++;
170      wakeup(&ticks);
171      release(&tickslock);
172    }
...
178  }
```

Interrupt while Locking

- What if an interrupt occurs while holding a lock?

```
$ vim kernel/sysproc.c

 67  uint64
 68  sys_pause(void)
 69  {
...
 76    acquire(&tickslock);
 77    ticks0 = ticks;
...
 85    release(&tickslock);
 86    return 0;
 87  }
```

CPU1

```
$ vim kernel/trap.c

164  void
165  clockintr()
166  {
167    if(cpuid() == 0){
168      acquire(&tickslock);
169      ticks++;
170      wakeup(&ticks);
171      release(&tickslock);
172    }
...
178  }
```

Interrupt while Locking

- What if an interrupt occurs while holding a lock?

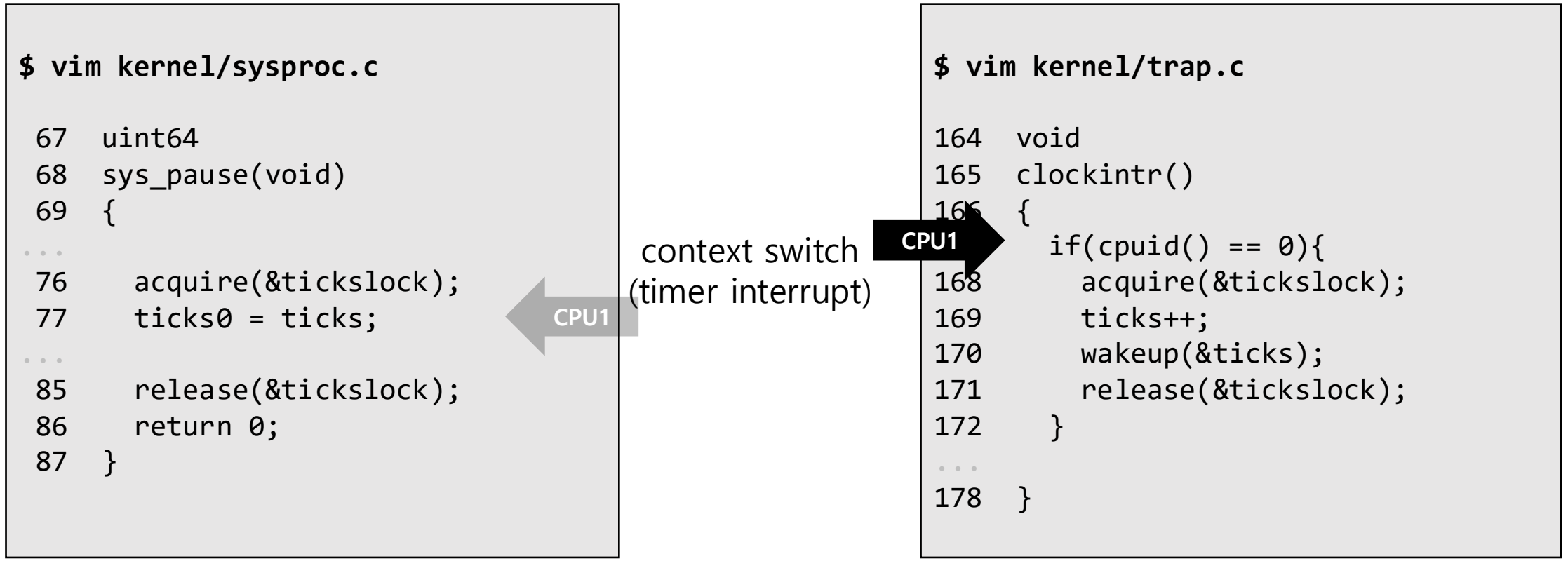
```
$ vim kernel/sysproc.c

 67  uint64
 68  sys_pause(void)
 69  {
...
 76    acquire(&tickslock);
 77    ticks0 = ticks;
...
 85    release(&tickslock);
 86    return 0;
 87  }
```

CPU1

```
$ vim kernel/trap.c

164  void
165  clockintr()
166  {
167    if(cpuid() == 0){
168      acquire(&tickslock);
169      ticks++;
170      wakeup(&ticks);
171      release(&tickslock);
172    }
...
178  }
```

Interrupt while Locking

- What if an interrupt occurs while holding a lock?



Interrupt while Locking

- What if an interrupt occurs while holding a lock?

```
$ vim kernel/sysproc.c

 67  uint64
 68  sys_pause(void)
 69  {
...
 76    acquire(&tickslock);
 77    ticks0 = ticks;
...
 85    release(&tickslock);
 86    return 0;
 87  }
```

CPU1

```
$ vim kernel/trap.c

164  void
165  clockintr()
166  {
167    if(cpuid() == 0){
         acquire(&tickslock);
169      ticks++;
170      wakeup(&ticks);
171      release(&tickslock);
172    }
...
178  }
```

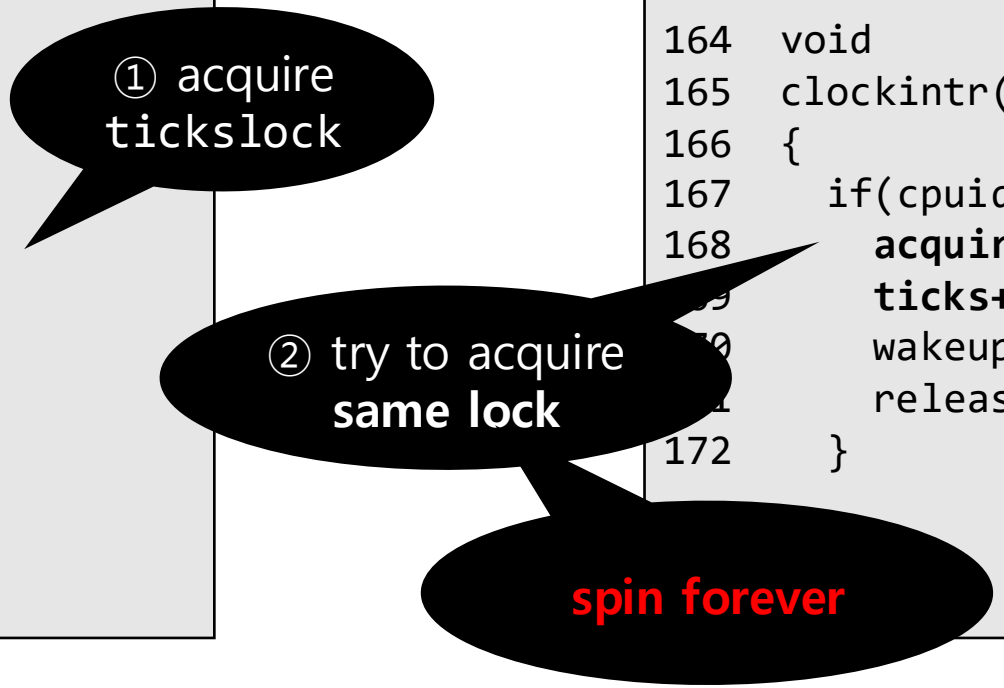
CPU1

Interrupt while Locking

- What if an interrupt occurs while holding a lock?

```
$ vim kernel/sysproc.c

 67  uint64
 68  sys_pause(void)
 69  {
...
 76    acquire(&tickslock);
 77    ticks0 = ticks;
...
 85    release(&tickslock);
 86    return 0;
 87  }
```

```
$ vim kernel/trap.c

164  void
165  clockintr()
166  {
167    if(cpuid() == 0){
168      acquire(&tickslock);
         ticks++;
         wakeup(&ticks);
         release(&tickslock);
172    }
```



Disable Interrupts : push_off / pop_off

```
$ vim kernel/spinlock.c

 21  void
 22  acquire(struct spinlock *lk)
 23  {
 24    push_off(); // disable interrupts to avoid deadlock.
 25    if(holding(lk))
 26      panic("acquire");
...
 32    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
 33      ;
...
 39    __sync_synchronize();
...
 42    lk->cpu = mycpu();
 43 }
```

```
$ vim kernel/spinlock.c

 46  void
 47  release(struct spinlock *lk)
 48  {
 49    if(holding(lk))
 50      panic("release");
 51
 52    lk->cpu = 0;
...
 60    __sync_synchronize();
...
 69    __sync_lock_release(&lk->locked);
 70
 71    pop_off();
 72  }
```

Nested Critical Section

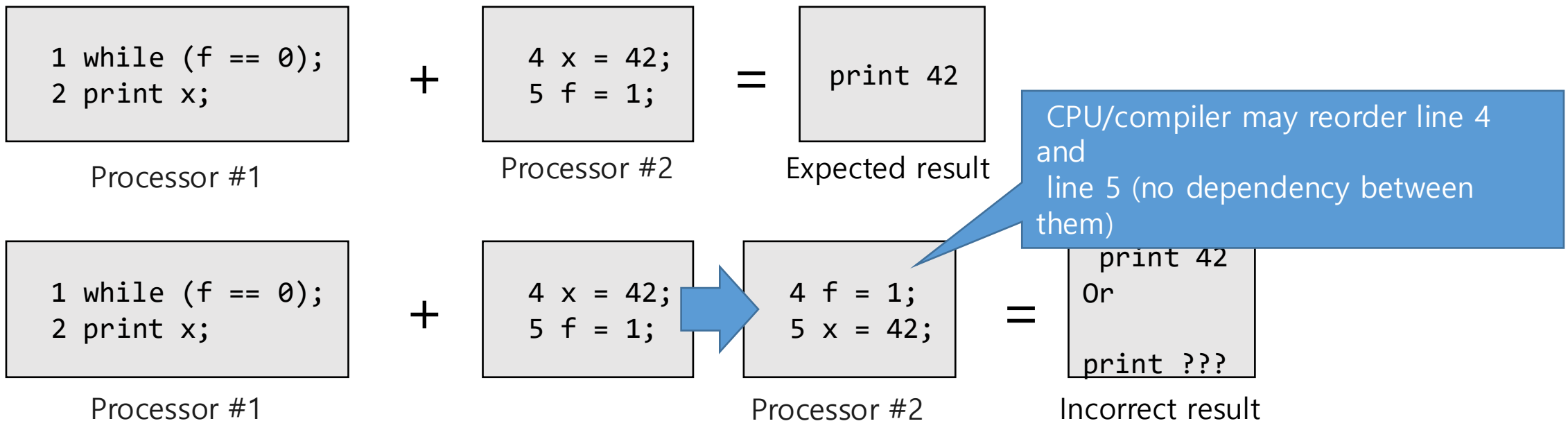
- xv6 re-enables interrupts only when a CPU holds **no** spinlocks.
- `push_off()` / `pop_off()` track the nesting level

```
$ vim kernel/spinlock.c

 88  void
 89  push_off(void)
 90  {
 91    int old = intr_get();
...
 95    intr_off();
 96    if(mycpu()->noff == 0)
 97      mycpu()->intena = old;
 98    mycpu()->noff += 1;
 99  }
```

```
$ vim kernel/spinlock.c

102  void
103  pop_off(void)
104  {
105    struct cpu *c = mycpu();
106    if(intr_get())
107      panic("pop_off - interruptible");
108    if(c->noff < 1)
109      panic("pop_off");
110    c->noff -= 1;
111    if(c->noff == 0 && c->intena)
112    intr_on();
113  }
```

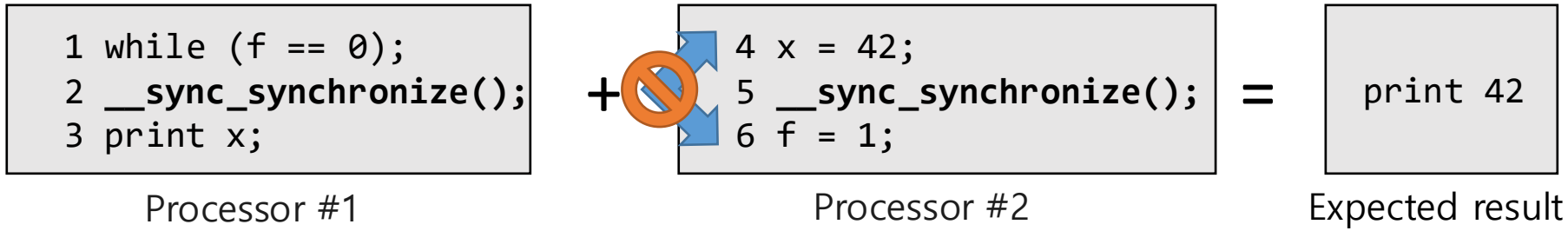
Memory Ordering : Problem

- Compilers and processors may execute instructions **out of order** for performance



Memory Ordering : Solution

- To tell the hardware and compiler not to perform such reorderings, xv6 uses `__sync_synchronize()` in both acquire and release.
- `__sync_synchronize()` is a **memory barrier**
  - Tells the compiler and CPU not to reorder loads/stores across the barrier.



Acquire with Memory Barrier

```
$ vim kernel/spinlock.c

 21  void
 22  acquire(struct spinlock *lk)
 23  {
 24    push_off(); // disable interrupts to avoid deadlock.
 25    if(holding(lk))
 26      panic("acquire");
...
 32    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
 33      ;
...
 39    __sync_synchronize();
...
 42    lk->cpu = mycpu();
 43  }
```

```
$ vim kernel/spinlock.c

 46  void
 47  release(struct spinlock *lk)
 48  {
 49    if(holding(lk))
 50      panic("release");
 51
 52    lk->cpu = 0;
...
 60    __sync_synchronize();
...
 69    __sync_lock_release(&lk->locked);
 70
 71    pop_off();
 72  }
```

Recap: Lock Implementation in xv6

```
$ vim kernel/spinlock.c

21  void
22  acquire(struct spinlock *lk)
23  {
24    push_off(); // disable interrupts to avoid deadlock.   ②
25    if(holding(lk))
26      panic("acquire");
...
32    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)   ①
33      ;
...
39    __sync_synchronize();   ③
...
42    lk->cpu = mycpu();
43  }
```

```
$ vim kernel/spinlock.c

46  void
47  release(struct spinlock *lk)
48  {
49    if(holding(lk))
50      panic("release");
51
52    lk->cpu = 0;
...
60    __sync_synchronize();   ③
...
69    __sync_lock_release(&lk->locked);   ①
70
71    pop_off();   ②
72  }
```

#	Component	Why
①	Atomic operation	Prevent two CPUs from acquiring the lock simultaneously
②	Interrupt disable	Prevent infinite wait during handling the kernel trap
③	Memory barrier	Prevent instruction reordering across loc k boundary

Key Considerations in Using Locks

Locks in Practice

Recall: Locks in scheduler

- In xv6 scheduling, `p->lock` is acquired in `yield()` and released in `scheduler()`

```
$ vim kernel/proc.c

493  void
494  yield(void)
495  {
496    struct proc *p = myproc();
497    acquire(&p->lock);
499    sched();
500    release(&p->lock);
501  }
```

context switch

```
$ vim kernel/proc.c

424  void
425  scheduler(void)
426  {
431    for(;;){
441      for(p = proc; p < &proc[NPROC]; p++)
442        acquire(&p->lock);
443        if(p->state == RUNNABLE) {
449          swtch(&c->context, &p->context);
455        }
456        release(&p->lock);
457      }
```

- Lock acquired by one context, released by another (across `swtch`)

Precautions When Using Multiple Locks

- Let's say two code paths in xv6 needs locks A and B.
- What happens when two CPUs run each code paths concurrently?

CPU1

```
1  void
2  function1()
3  {
4    acquire(&A.lock);
5    acquire(&B.lock);
6    // execute critical section code
7    release(&B.lock);
8    release(&A.lock);
9  }
```

CPU2

```
11 void
12 function2()
13 {
14   acquire(&B.lock);
15   acquire(&A.lock);
16   // execute critical section code
17   release(&A.lock);
18   release(&B.lock);
19 }
```

Precautions When Using Multiple Locks

- Let's say two code paths in xv6 needs locks A and B.
- What happens when two CPUs run each code paths concurrently?

CPU1

```
1  void
2  function1()
3  {
4    acquire(&A.lock);
5    acquire(&B.lock);
6    // execute critical section code
7    release(&B.lock);
8    release(&A.lock);
9  }
```

CPU2

```
11 void
12 function2()
13 {
14   acquire(&B.lock);
15   acquire(&A.lock);
16   // execute critical section code
17   release(&A.lock);
18   release(&B.lock);
19 }
```

Precautions When Using Multiple Locks

- Let's say two code paths in xv6 needs locks A and B.
- What happens when two CPUs run each code paths concurrently?

CPU1

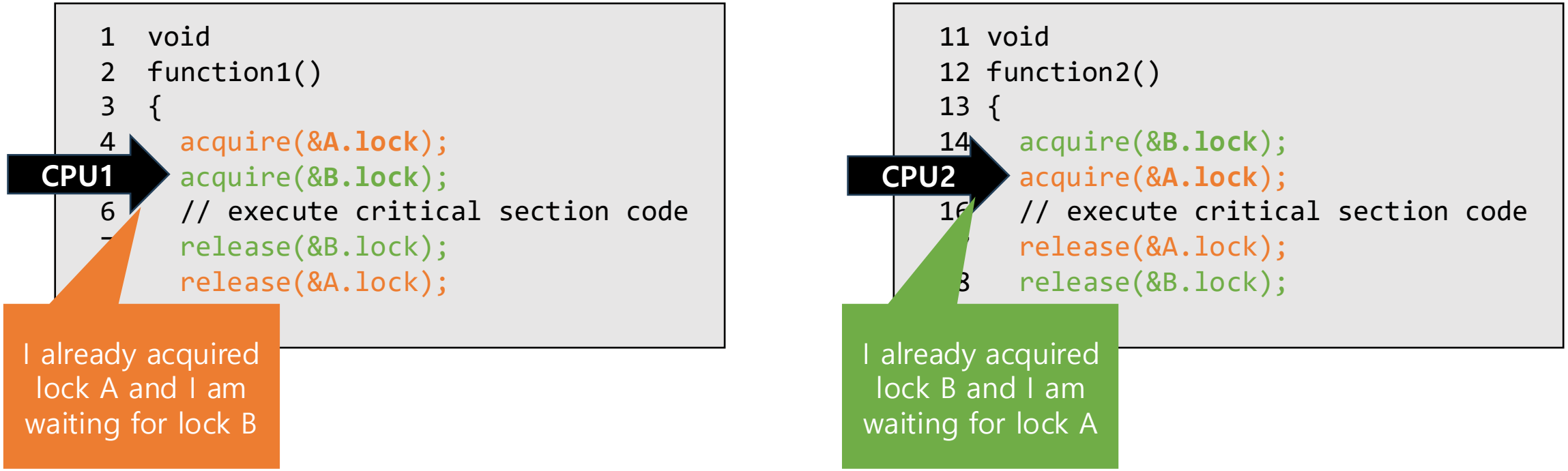
```
1  void
2  function1()
3  {
4    acquire(&A.lock);
5    acquire(&B.lock);
6    // execute critical section code
7    release(&B.lock);
8    release(&A.lock);
9  }
```

CPU2

```
11 void
12 function2()
13 {
14   acquire(&B.lock);
15   acquire(&A.lock);
16   // execute critical section code
17   release(&A.lock);
18   release(&B.lock);
19 }
```

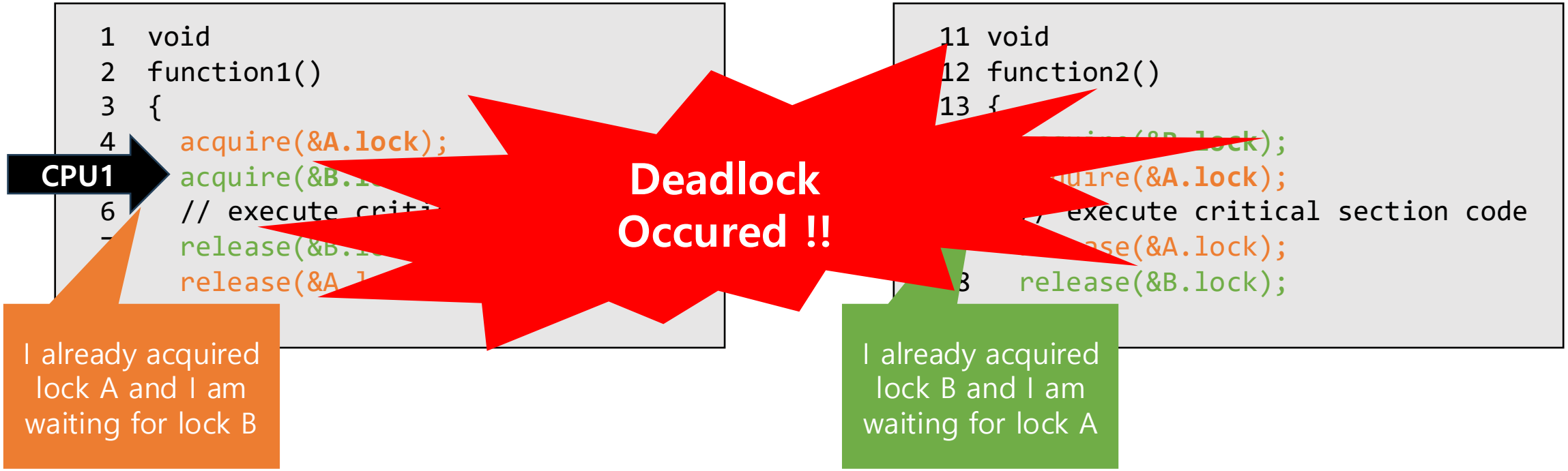
Precautions When Using Multiple Locks

- Let's say two code paths in xv6 needs locks A and B.
- What happens when two CPUs run each code paths concurrently?



Precautions When Using Multiple Locks

- Let's say two code paths in xv6 needs locks A and B.
- What happens when two CPUs run each code paths concurrently?



Deadlock

- This creates a **circular wait**
  - Function1 holds lock A and waits for lock B
  - Function2 holds lock B and waits for lock A
- Neither function can proceed
  - This situation is called a deadlock

Precautions When Using Multiple Locks

- All code paths must acquire locks in the **same order**

```
1  void
2  function1()
3  {
4    acquire(&A.lock);
5    acquire(&B.lock);
6    // execute critical section code
7    release(&B.lock);
8    release(&A.lock);
9  }
```

```
11  void
12  function2()
13  {
14    acquire(&A.lock);
15    acquire(&B.lock);
16    // execute critical section code
17    release(&B.lock);
18    release(&A.lock);
19  }
```

- xv6 maintains a global lock acquisition order to prevent deadlock.
  - consoleintr : cons.lock → process lock
  - File creation : directory → inode → disk block →vdisk_lock → process lock

Locks in Uniprocessor System

```
$ vim kernel/proc.c

592  int
593  kkill(int pid)
594  {
...
597    for(p = proc; p < &proc[NPROC]; p++){
598      acquire(&p->lock);
599      if(p->pid == pid){
600        p->killed = 1;
601        if(p->state == SLEEPING){
602          // Wake process from sleep().
603          p->state = RUNNABLE;
604        }
605        release(&p->lock);
606        return 0;
607      }
608      release(&p->lock);
609    }
610    return -1;
611  }
```

- In a uniprocessor system, do we still need locks?

Locks in Uniprocessor System

```
$ vim kernel/proc.c

592  int
593  kkill(int pid)
594  {
...
597    for(p = proc; p < &proc[NPROC]; p++){
598      acquire(&p->lock);
599      if(p->pid == pid){
600        p->killed = 1;
601        if(p->state == SLEEPING){
602          // Wake process from sleep().
603          p->state = RUNNABLE;
604        }
605        release(&p->lock);
606        return 0;
607      }
608      release(&p->lock);
609    }
610    return -1;
611  }
```

- In a uniprocessor system, do we still need locks?

→ **YES**

Locks in Uniprocessor System

```
$ vim kernel/proc.c

592  int
593  kkill(int pid)
594  {
...
597    for(p = proc; p < &proc[NPROC]; p++){
598      acquire(&p->lock);
599      if(p->pid == pid){
600        p->killed = 1;
601        if(p->state == SLEEPING){
602          // Wake process from sleep().
603          p->state = RUNNABLE;
604        }
605        release(&p->lock);
606        return 0;
607      }
608      release(&p->lock);
609    }
610    return -1;
611  }
```

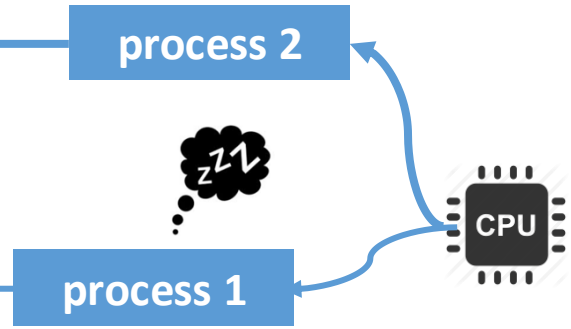
Locks in Uniprocessor System

```
$ vim kernel/proc.c

592  int
593  kkill(int pid)
594  {
...
597    for(p = proc; p < &proc[NPROC]; p++){
598      acquire(&p->lock);
599      if(p->pid == pid){
600        p->killed = 1;
601        if(p->state == SLEEPING){
602          // Wake process from sleep().
603          p->state = RUNNABLE;
604        }
605        release(&p->lock);
606        return 0;
607      }
608      release(&p->lock);
609    }
610    return -1;
611  }
```

Timer Interrupt occurs!

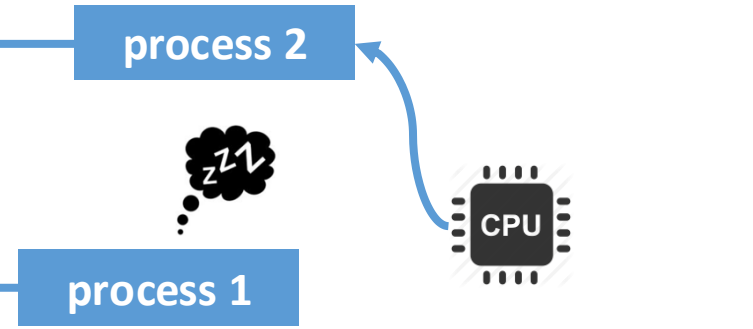
Locks in Uniprocessor System

```
$ vim kernel/proc.c

592  int
593  kkill(int pid)
594  {
...
597    for(p = proc; p < &proc[NPROC]; p++){
598      acquire(&p->lock);
599      if(p->pid == pid){
600        p->killed = 1;
601        if(p->state == SLEEPING){
602          // Wake process from sleep().
603          p->state = RUNNABLE;
604        }
605        release(&p->lock);
606        return 0;
607      }
608      release(&p->lock);
609    }
610    return -1;
611  }
```



Locks in Uniprocessor System

```
$ vim kernel/proc.c

592  int
593  kkill(int pid)
594  {
...
597    for(p = proc; p < &proc[NPROC]; p++){
598      acquire(&p->lock);
599      if(p->pid == pid){
600        p->killed = 1;
601        if(p->state == SLEEPING){
602          // Wake process from sleep().
603          p->state = RUNNABLE;
604        }
605        release(&p->lock);
606        return 0;
607      }
608      release(&p->lock);
609    }
610    return -1;
611  }
```



Race condition occurs!

Locks in Uniprocessor System

```
$ vim kernel/proc.c

592  int
593  kkill(int pid)
594  {
...
597    for(p = proc; p < &proc[NPROC]; p++){
598      acquire(&p->lock);
599      if(p->pid == pid){
600        p->killed = 1;
601        if(p->state == SLEEPING){
602          // Wake process from sleep().
603          p->state = RUNNABLE;
604        }
605        release(&p->lock);
606        return 0;
607      }
608      release(&p->lock);
609    }
610    return -1;
611  }
```

- In a uniprocessor system, do we still need locks?

  → **YES**

- Even in a uniprocessor system, race conditions can occur via **interrupt-driven context switches**
- Locks (with interrupt disable) are still necessary

Hand-on Labs

Implementing FCFS Scheduling

Branch Setup

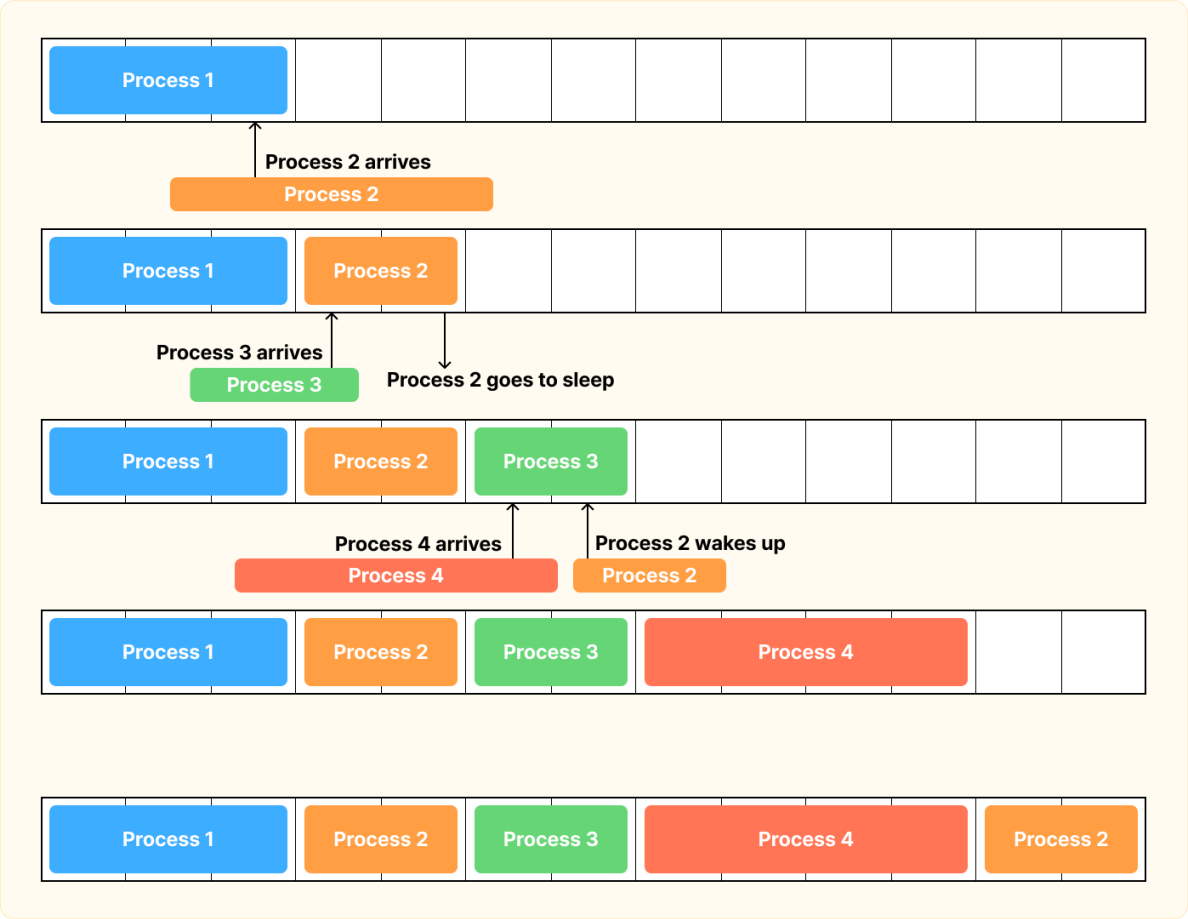
- Make a branch named 'sandbox/lab06' in sandbox repository

```
$ cd <path_to_your_own_sandbox_repo>    # move to your sandbox repository
$ git switch sandbox/base               # switch to base branch
$ git branch sandbox/lab06              # make a new branch
$ git switch sandbox/lab06              # switch to lab05 branch
```

- TA's code will be uploaded in **xv6-riscv-sandbox** repository on **next weekends**.

Implementing FCFS Scheduling

- **Goal:** Replace xv6's default Round Robin scheduler with a First-Come, Frist-Serve (FCFS) Scheduler and test codes.



User Tests for FCFS Scheduling

- The full test code is available on the link.

```
$ vim user/usertests.c

3153  fcfs_test1(char *s)
...
3160    char a_char = 'a';
3161    for(n=0; n<N; n++){
3162      pid = fork();
3163      if(pid < 0) {
3164        break;
3165      } else if (pid == 0) {
3166        for (long i = 0; i < RUN_CNT; i++) {
3167          if (i % STEP_CNT == 0) {
3168            printf("%c", (a_char + (char) n));
3169          }
3170        }
3171        printf("\n");
3172        exit(0);
3173      }
3174    }
3175
3176    for(n=0; n<N; n++){
3177      wait(0);
3178    }
...
```

```
3184  fcfs_test2(char *s)
...
3191    char a_char = 'a';
3192    for(n=0; n<N; n++){
3193      pid = fork();
3194      if(pid < 0) {
3195        break;
3196      } else if (pid == 0) {
3197        for (int i = 0; i < YEILD_CNT; i++) {
3198          for (long j = 0; j < RUN_CNT; j++) {
3199            if (j % STEP_CNT == 0) {
3200              printf("%c", (a_char + (char) ((i * N) + n)));
3201            }
3202          }
3203          printf("\n");
3204          pause(10);
3205        }
3206        exit(0);
3207      }
3208    }
3209    for(n=0; n<N; n++){
3210      wait(0);
3211    }
...
```

Data Structure for FCFS: Circular Queue

- A Queue data structure is required to implement the FCFS scheduler.
- The full implementation code is available on the link.

```
$ vim kernel/queue.h

...
 8  struct queue {
 9    struct spinlock lock;
10
11    int head;
12    int tail;
13
14    void *data[MAX_QUEUE_SIZE + 1];
15  };
```

```
$ vim kernel/defs.h

...
185  // queue.c
186  void queue_init(struct queue *);
187  int queue_push(struct queue *, void *);
188  void * queue_peek(struct queue *);
189  void * queue_pop(struct queue *);
190  int queue_size(struct queue *);
```

- **Modify the scheduler code in kernel/proc.c using this queue implementation.**

Submission

- Files to change

```
$ git status --short

 M Makefile
 M kernel/defs.h
 M kernel/proc.c
 M user/usertests.c
?? kernel/queue.c
?? Kernel/queue.h
```

- Commit and push to your own remote repository

```
$ git add .
$ git commit –m “lab06: implement FCFS scheduling”
$ git push origin sandbox/lab06
```

Thank you